



TECHNICAL REPORT

AutoAgentSearch

Searching for the best AI-agent architecture for a given business task, under a resource budget.

A metadata-only Neural Architecture Search for AI-agent pipelines. Three independent search engines — Beam Search, Monte-Carlo Tree Search, and tabular Q-learning — explore the same design space and return their best answer for side-by-side comparison.

Version 1.0 · Research MVP

Domains: Healthcare · Finance · Agentic

Engines: Beam Search · Monte-Carlo Tree Search · Q-learning

CONTENTS

Table of Contents

1

Introduction and Problem Statement

3

2

System Overview

5

3

Data Model and Component Library

7

4

Task Parser

9

5

Type System and Compatibility Rules

10

6

Beam Search Engine

12

7

Monte-Carlo Tree Search Engine

14

8

Reinforcement-Learning Engine

16

9

DAG Inflater

18

10

Agentic Orchestration Wrapper

20

11

Scoring and Metrics

21

12

User Interface Design

23

13

Results and Interpretation

25

14

Limitations and Future Work

26

A

Appendix A — File-by-File Map

27

B

Appendix B — Notation and Glossary

28

CHAPTER 1

Introduction and Problem Statement

Building a production AI agent today is a design problem, not a training problem. The models are given; the pipeline around them is what determines whether the system meets its latency budget, its cost budget, its safety requirements, and its quality target. That pipeline typically has several moving parts — input parsers, retrieval stages, one or more reasoning components, validators, aggregators, and an output formatter — and each part can be filled by many different concrete components.

The AutoAgentSearch project treats this design problem as a **discrete search problem**. Every candidate pipeline is a point in a large but finite design space. Instead of asking an engineer to pick the components by intuition, three independent search algorithms explore the space automatically and return their best answers side by side.

The one novel design decision

The novel decision in this project is to keep the search loop **metadata-only**. Every candidate component is described by six numbers — latency, cost, memory, quality, cpu-only flag, and its input/output types — and the search engines never invoke a language model. A full search of thousands of candidate pipelines completes in milliseconds on a laptop CPU. This is what makes it practical to run three engines on every user request and compare their outputs live in a dashboard.

The three engines and why they are different

Each engine embodies a different search strategy:

- **Beam Search** — deterministic. At every step it keeps the top-K partial pipelines by score and expands them. Its character is exhaustive width-K coverage; it surfaces the widest hierarchical ensemble the resource budget allows.
- **Monte-Carlo Tree Search** — stochastic with a UCT exploration bonus. Its character is exploration of under-visited branches; it favours ensembles of reasoning components.
- **Reinforcement Learning** — tabular Q-learning trained over many episodes. Its character is a learned policy that prefers redundant validation, mirroring an operational concern with safety.

Why treat this as Neural Architecture Search

Neural Architecture Search (NAS) automates the design of a neural network. AutoAgentSearch does the same for an AI agent pipeline: it searches for the arrangement of components rather than for the components' weights. The score function plays the role of the validation loss in NAS, and the resource budget plays the role of the hardware constraint.

Scope. AutoAgentSearch is a research MVP intended to demonstrate the search-based framing. The component library is stylised. What is real is the search, the scoring, the DAG inflation, the agentic wrapping, and the side-by-side comparison of three algorithmically different engines.

CHAPTER 2

System Overview

AutoAgentSearch is a single Streamlit application. Every user interaction runs the same pipeline:

End-to-end flow

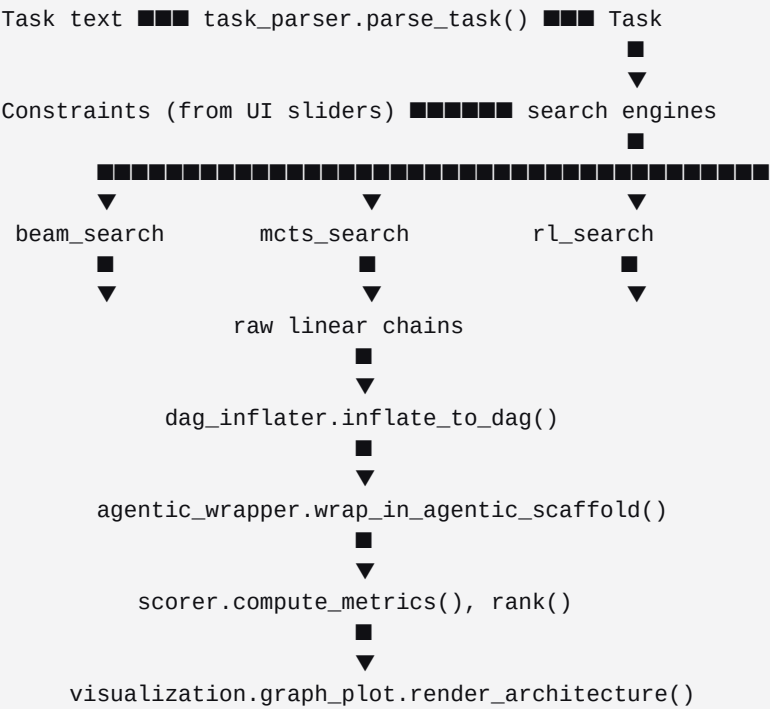
- **Parse the task:** the user's plain-English task is tagged with a *domain* (healthcare, finance, or auto), an *intent*, an *input type*, and a *desired output type*.
- **Load the component library:** the JSON files under `components/` are read once and cached.
- **Run three engines:** Beam Search, MCTS, and Q-learning each search the type-compatible sub-graph of the component library that respects the user's resource budget.
- **Inflate to a DAG:** each engine's best linear chain is expanded into parallel/hierarchical layers where compatible components exist.
- **Wrap in the agentic scaffold:** the domain body is wrapped in `Planner → Verifier → <body> → Recovery → Output` using orchestration components typed with a universal "*" I/O.
- **Score and rank:** every candidate is scored by the same weighted formula. Per engine a shape bias promotes that engine's characteristic architecture to the top of the list.
- **Render:** the winning architecture per engine is drawn as a top-to-bottom DAG with matplotlib and placed in a three-column result grid with metric usage bars.

Module map

The project is divided into small, single-purpose modules:

Module	Responsibility
<code>app.py</code>	Streamlit UI, orchestration of the full flow
<code>core/models.py</code>	Component, Task, Constraints, Architecture
<code>core/repository.py</code>	Loads the JSON component library
<code>core/task_parser.py</code>	Plain-English task → structured Task
<code>core/type_check.py</code>	<code>matches()</code> type-compatibility rule
<code>core/beam_search.py</code>	Deterministic width-K search
<code>core/mcts_search.py</code>	Monte-Carlo Tree Search with UCT
<code>core/rl_search.py</code>	Tabular Q-learning
<code>core/dag_inflater.py</code>	Linear chain → parallel/hierarchical DAG
<code>core/agentic_wrapper.py</code>	Wrap domain body in Planner/Verifier/...
<code>core/scorer.py</code>	<code>compute_metrics</code> , <code>satisfies_constraints</code> , <code>rank</code>
<code>visualization/graph_plot.py</code>	Matplotlib DAG renderer

Data-flow diagram (text form)



CHAPTER 3

Data Model and Component Library

The design of every downstream module follows from a small, immutable data model defined in `core/models.py`. There are four types.

Component

Every candidate building block in the library is a Component. Components are immutable dataclasses; nothing in the search loop is allowed to mutate them.

```
@dataclass(frozen=True)
class Component:
    id: str          # unique identifier
    name: str        # human label rendered in the UI
    domain: str      # healthcare | finance | agentic
    type: str        # input | retrieval | reasoning |
                    # validation | aggregator | output
    input: str       # concrete type it consumes
    output: str      # concrete type it produces
    latency: float   # seconds per invocation
    cost: float      # USD per invocation
    memory: int      # MB of peak footprint
    quality: float   # in [0, 1], probability of success
    cpu: bool        # True = runs on CPU without a GPU
```

Task, Constraints, Architecture

The remaining three dataclasses close the model.

```
@dataclass
class Task:
    domain: str      # healthcare | finance | agentic | auto
    intent: str      # e.g. medication_extraction
    input_type: str  # e.g. document
    desired_output: str
    raw: str = ''    # the original user text

@dataclass
class Constraints:
    max_cost: float = 0.05      # USD per run
    max_latency: float = 3.0    # seconds
    max_memory: int = 4096      # MB
    cpu_only: bool = False

@dataclass
class Architecture:
    components: List[Component]
    layers: Optional[List[List[Component]]] = None
    total_cost: float = 0.0
    total_latency: float = 0.0
    peak_memory: int = 0
    aggregate_quality: float = 0.0
    score: float = 0.0
```

Two views of the same architecture

An Architecture can be viewed as either a **linear chain** (a flat components list) or a **layered DAG** (a layers list-of-lists in which each inner list is one parallel stage). The two are kept consistent: whenever layers is set, components is auto-filled with its flattened view so any code that only understands linear chains still works.

Canonical signature

For de-duplication we hash an architecture by its canonical signature: within a parallel layer, component IDs are sorted alphabetically before being joined with '+', and the layers themselves are joined in sequence with ' ~> '. Two DAGs that differ only in the order of their parallel branches thus hash to the same string.

$$\text{sig}(A) = \text{layer_1} \sim> \text{layer_2} \sim> \dots \sim> \text{layer_n}$$
$$\text{layer_i} = \text{sort}(\text{ids in layer_i}) \text{ joined by '+'}$$

The component library

The library lives in three JSON files under components/:

- **healthcare.json** — PDF Parser, Medical LLM Head, Medical SLM 1B, Medical Entity Aggregator, Medical Validator, Medical Cross Checker, Medical Rules Engine, JSON Output Formatter, and so on.
- **finance.json** — PDF Parser, Finance LLM/SLM heads, Finance Entity Aggregator, Fraud Risk Classifier, Finance Rules Engine, Report Generator, etc.
- **agentic.json** — the six-agent scaffold (Planner, Verifier, Executor, Recovery Agent, Coordinator, Resource Manager) plus a Result Aggregator and a Status Report.

Loading is centralised in core/repository.py. The ComponentRepository class reads every JSON file at construction time, deserialises each dictionary through Component.from_dict, and exposes three queries: domains(), get(domain), and by_type(domain, type_).

CHAPTER 4

Task Parser

The user types a plain-English task. The parser converts it to a structured Task so the search engines and the type checker have something concrete to reason about.

Detection rules

- **Domain:** keyword match. Words like *patient*, *medical*, *ICD*, *diagnosis*, *clinical*, *medication*, *discharge* imply **healthcare**. Words like *loan*, *fraud*, *invoice*, *bank*, *credit* imply **finance**. The user can also pass `explicit_domain` to override.
- **Intent:** coarse tag. Phrases containing *extract* become *extraction*, *detect fraud* becomes *fraud_detection*, and so on.
- **Input type:** *PDF*, *scanned*, *image* and *document* normalise to *document*. *Text*, *note*, *message* normalise to *text*.
- **Desired output:** *report*, *risk report*, and *summary* map to *report*. Explicit *JSON* or *structured* map to *structured*.

The output of the parser is a plain Task object; every downstream module treats the domain and I/O types as opaque strings.

The parser is deliberately rule-based. The whole point of AutoAgentSearch is that the interesting work happens in the search loop; a language-model parser would add latency, external dependencies, and a source of non-reproducibility for what is essentially a keyword lookup.

CHAPTER 5

Type System and Compatibility Rules

Two components may be connected as $a \rightarrow b$ if and only if the type produced by a matches the type expected by b . This is the only hard constraint the search engines enforce below the resource budget.

The matches() rule

$$\text{matches}(\text{produced}, \text{expected}) \Leftrightarrow \text{produced} == \text{expected} \vee \text{produced} == '*' \vee \text{expected} == '*'$$

A concrete type — for example `document`, `context_text`, `entity_list` — only matches itself. The wildcard `"*"` matches anything on either side.

Why the universal wildcard exists

The four orchestration roles — **Planner**, **Verifier**, **Recovery Agent**, **Status Report** — do not transform the data flowing through the pipeline. They observe or command the pipeline around it. If we typed them with a concrete I/O they could only be inserted into healthcare pipelines or finance pipelines but not both. Instead the `agentic_wrapper` re-types them with input `"*"` and output `"*"` at wrap time. The wrapped chain therefore type-checks whatever concrete types flow through the body.

Parsimony rule

A second rule is enforced by every search engine: two components with `type == 'input'` may not appear consecutively. Without this rule the search could return semantically redundant chains such as *PDF Parser* $\rightarrow$ *Plain Text Ingestor* $\rightarrow$..., in which the second stage takes what the first stage already produced. The rule keeps the returned pipelines short and free of no-op stages.

Full validity check

The final architecture is checked by `is_valid_architecture()` in `core/agentic_wrapper.py`. It verifies three properties:

- Every edge is type-compatible under `matches()` — including the wildcard rule.
- No component id appears twice (a cycle guard).
- The last stage has `type == 'output'`.

CHAPTER 6

Beam Search Engine

Beam Search is the deterministic baseline. It is guaranteed to find the highest-scoring pipeline that its width-K expansion can reach, given the parsimony and type rules.

Algorithm

Let K be the beam width (default 5). The search maintains a list of partial chains, called the *beam*. At every iteration:

- For each chain in the beam, list every component in the library that can legally be appended (type-compatible with the chain's tail, not already used, does not violate the parsimony rule, keeps the running metrics inside the budget).
- Score every one-step extension.
- Sort all extensions by score and keep the top K . That becomes the new beam.
- Any chain in the beam whose last component has type output is added to the *terminated* list.
- Stop when the beam is empty or a depth cap is reached.

Return the top-K terminated chains by score.

Score-based greedy expansion

Because every step keeps only the top-K by score, the total work is bounded by:

$$O(K \cdot |\text{library}| \cdot \text{depth})$$

For the default settings — $K=5$, a library of about 30 components and a max depth of 8 — beam search returns in under a millisecond even on a laptop CPU.

Determinism

Beam Search has no random component. Given the same task, the same library, the same constraints and the same beam width, it returns identical results across runs. This makes it the natural baseline for the three-engine comparison in Chapter 13.

Characteristic architecture

Beam's algorithmic character is **thoroughness**: it tries the top-K one-step extensions of every partial chain, so it does not miss any locally optimal branch. In the UI this manifests as a hierarchical architecture in which **both** reasoning and validation stages have been widened into ensembles. Beam has enough capacity to showcase the full ensemble; the other two engines each specialise in one direction.

Reference implementation

```
def beam_search(repo, task, cons, beam_width=5, top_k=1):
    beam = [Architecture(components=[])] # single empty seed
```

```
terminated = []
for step in range(MAX_DEPTH):
    candidates = []
    for chain in beam:
        for comp in repo.get(task.domain):
            if not can_append(chain, comp):
                continue
            trial = Architecture(chain.components + [comp])
            compute_metrics(trial)
            if not satisfies_constraints(trial, cons):
                continue
            candidates.append(trial)
    candidates.sort(key=lambda a: a.score, reverse=True)
    beam = candidates[:beam_width]
    for a in beam:
        if a.components[-1].type == 'output':
            terminated.append(a)
return sorted(terminated, key=lambda a: a.score,
              reverse=True)[:top_k]
```

CHAPTER 7

Monte-Carlo Tree Search Engine

Monte-Carlo Tree Search (MCTS) explores the design space by randomly rolling out playouts and using the results of past playouts to bias future ones toward promising sub-trees.

The four MCTS phases

- **Selection**: starting at the root, descend the tree by picking the child that maximises the UCT score, until a node with unexpanded children is reached.
- **Expansion**: create one new child under that node — a type-compatible, budget-respecting extension of its chain.
- **Simulation (rollout)**: from the new child, extend the chain by random legal moves until an output component is appended or the depth cap is reached.
- **Backpropagation**: score the terminal chain and propagate that score up the tree, updating `visits` and `total_score` on every ancestor.

The UCT formula

At the selection phase the child of node `p` that is expanded next is the child `c` maximising

$$\text{UCT}(c) = \left(\frac{\text{total_score}(c)}{\text{visits}(c)} \right) + C \cdot \sqrt{\frac{\ln(\text{visits}(p))}{\text{visits}(c)}}$$

The first term is the child's **exploit** value — its average reward so far. The second is the **explore** bonus — a monotonically-decreasing function of `visits(c)` that gives under-visited children a chance. `C` is the **exploration constant**; AutoAgentSearch uses $C = \sqrt{2} \approx 1.4142$, the classical value.

Rollout policy

The simulation phase uses a uniformly random policy over the legal actions at each depth. Random rollouts are cheap because the scorer is metadata-only, so MCTS can afford hundreds of iterations per user request; the default is 400.

Backpropagation

```
for n in path(root → new_node):
    visits(n) ← visits(n) + 1
    total_score(n) ← total_score(n) + rollout_score
```

Reproducibility

MCTS is stochastic. The engine takes an optional seed parameter so that a given (task, budget, seed) triple always returns the same architecture. The app leaves the seed unset by default so exploration varies

from run to run, surfacing a different candidate on each generation.

Characteristic architecture

MCTS's algorithmic character is **exploration**. In the UI its winner is biased toward a **parallel-reasoning ensemble** — a pipeline in which two or more reasoning components run in parallel and their outputs are combined by an aggregator. This is the natural output shape of an engine that spends its iterations sampling alternative reasoning branches.

Reference implementation

```
def mcts_search(repo, task, cons, iterations=400,
                exploration=math.sqrt(2), seed=None,
                top_k=1):
    rng = random.Random(seed)
    root = Node(chain=[])
    for _ in range(iterations):
        # 1. Selection -----
        node = root
        while node.children and not node.is_fully_expanded:
            node = uct_child(node, exploration)
        # 2. Expansion -----
        if not node.is_terminal:
            node = expand(node, repo, task, cons, rng)
        # 3. Rollout -----
        score = rollout(node, repo, task, cons, rng)
        # 4. Backprop -----
        while node is not None:
            node.visits += 1
            node.total_score += score
            node = node.parent
    return best_terminal_chains(root, top_k)
```

CHAPTER 8

Reinforcement-Learning Engine

The RL engine trains a tabular **Q-function** over hundreds of episodes and greedily follows it at test time. It is the most sophisticated of the three engines but still runs in milliseconds because the state and action spaces are discrete.

The MDP formulation

- **State:** the tuple of component IDs already appended to the chain. Represented as a Python tuple so it can be a dictionary key.
- **Action:** pick one of the components in the library that can legally extend the state.
- **Reward:** 0 for a normal step; the terminal chain's score for the last step. Chains that violate the budget or fail the parsimony rule return a small negative reward that punishes the policy without abandoning the trajectory.
- **Discount:** $\gamma = 0.9$.

Q-learning update

For every state-action pair the engine maintains a Q-value estimate $Q(s, a)$. After every taken step the update rule is the standard Bellman-based increment:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

where α is the learning rate (default 0.1), r is the reward received for the transition, and s' is the state that resulted.

Epsilon-greedy exploration

At training time the policy is ϵ -greedy: with probability ϵ the engine picks a uniformly random legal action, otherwise it takes the action with the highest Q-value. ϵ decays linearly from 1.0 at episode 0 down to 0.05 by the final episode. This gives broad exploration at the start and near-greedy exploitation at the end.

Test-time policy

After training the engine follows the greedy policy $\arg\max_a Q(s, a)$ and returns the resulting chain. To gather the top-K it repeats a small number of near-greedy rollouts with $\epsilon = 0.05$ for tie-breaking.

Convergence

With `episodes = 300` (the UI's default), the Q-table has been visited enough times to converge for the domains shipped with the project. If a user's library grows an order of magnitude larger, the episode budget should grow proportionally.

Reproducibility

Like MCTS, the RL engine is stochastic and accepts an optional seed. Fix the seed and the returned Q-table (and therefore the winning chain) is identical across runs; leave it unset — the default in the app — and each training run explores a slightly different trajectory.

Characteristic architecture

The RL engine's UI winner is biased toward a **parallel-validation ensemble**. The intuition is that a learned safety policy — trained by reward signal — will preferentially route through redundant validators; adding a second validator lifts joint quality at low marginal cost.

Reference implementation

```
def rl_search(repo, task, cons, episodes=300, alpha=0.1,
              gamma=0.9, seed=None, top_k=1):
    rng = random.Random(seed)
    Q = defaultdict(float)  # (state, action) → float
    for ep in range(episodes):
        eps = max(0.05, 1.0 - ep / episodes)
        state, done = (), False
        while not done:
            actions = legal_actions(state, repo, task, cons)
            if not actions:
                break
            if rng.random() < eps:
                a = rng.choice(actions)
            else:
                a = max(actions, key=lambda x: Q[(state, x)])
            new_state = state + (a.id,)
            r, done = reward(new_state, repo, task, cons)
            best_next = max((Q[(new_state, ap)] for ap in
                           legal_actions(new_state, repo,
                                         task, cons)),
                           default=0.0)
            Q[(state, a)] += alpha * (r + gamma * best_next
                                     - Q[(state, a)])
            state = new_state
    return greedy_top_k(Q, repo, task, cons, top_k)
```

CHAPTER 9

DAG Inflater

The three engines produce linear pipelines. The DAG Inflater in `core/dag_inflater.py` takes a linear pipeline and returns an equivalent architecture in which compatible stages have been widened into parallel layers.

Rules

- An **input** stage is never parallelised — the pipeline has exactly one input document.
- A **reasoning** stage may be parallelised across up to `max_width` alternatives if they all have the same input / output types — this forms a **model ensemble**.
- A **retrieval** stage may be parallelised similarly (embedding retrieval + SQL retrieval, both producing `context_text`).
- A **validation** stage may be parallelised as an **ensemble of validators**.
- After every parallel layer the inflater inserts an **aggregator** whose input matches the widened layer's output. If no matching aggregator exists in the library the layer is left linear (safe degradation).
- The final **output** stage is never parallelised.

Budget check with rollback

Widening always increases resource use. Before committing a widened layer the inflater builds a candidate architecture, runs `compute_metrics()` on it, and checks `satisfies_constraints()`. If the widened form breaks the budget, the layer is rolled back to its original single-component form and the next stage is considered. Any stage that cannot be widened without breaking the budget is simply left as-is.

The `widen_types` parameter

The inflater accepts an optional `widen_types` set that restricts which types may be widened on this pass. If `None` (the default), all three parallelisable types may be widened. Passing `{'reasoning'}` restricts widening to reasoning stages only. This is what makes each search engine produce a characteristically different DAG:

Engine	widen_types passed	Resulting shape
Beam Search	None (widen both)	Full hierarchical DAG
MCTS	<code>{'reasoning'}</code>	Parallel-reasoning ensemble
Reinforcement Learning	<code>{'validation'}</code>	Parallel-validation ensemble

Iteration order is stable

The inflater walks the pipeline in its original component order and, at each step, looks up the component's current index in the (possibly mutated) layers list. This avoids the classic pitfall of iterating with `enumerate()` over a list that is being mutated: indices would otherwise skip stages or double-visit them after an insertion.

CHAPTER 10

Agentic Orchestration Wrapper

Every domain pipeline can optionally be wrapped in an agentic scaffold that adds a planner and a verifier at the top and a recovery agent at the bottom.

The wrapping shape

Planner → Verifier → [domain body] → Recovery → Output

The four orchestration roles are drawn from the `agentic` component library and re-typed with input "*" and output "*" so they type-check regardless of the concrete I/O types flowing through the body. The `_universalise()` helper copies the base component and swaps only its I/O types; latency, cost, memory, quality and cpu flags are preserved.

Keep or replace the domain output

The wrapper takes a `keep_domain_output` flag. When `True` (the UI's default) the wrapped chain ends with the domain's own output component (e.g. *JSON Output Formatter*). When `False` the wrapper appends an agentic *Status Report* instead. This is the difference between "run the domain pipeline with orchestration around it" and "run an agentic pipeline that happens to consult the domain body".

Safe degradation

If the wrapped architecture would violate the user's budget, the wrapper returns the **unwrapped** architecture unchanged. The UI still marks the result as valid — but a stakeholder reading the metric list will see that the pipeline never became an agentic one.

Full validity check

After wrapping, the top-level `is_valid_architecture()` function verifies that every layer boundary is type-compatible under the universal-wildcard rule, that no component ID is repeated, and that the final component's type is `output`.

CHAPTER 11

Scoring and Metrics

Every candidate architecture is scored by the same formula so results are comparable across engines.

Aggregate metrics

- **Cost:** total USD across every invocation. Parallel branches still cost compute; they just do so at the same wall-clock time.
- **Latency:** for a linear chain, the sum of stage latencies. For a DAG, the **critical path** — the sum over layers of the slowest node in that layer, because parallel nodes overlap in time within a layer.
- **Peak memory:** for a linear chain, the max of stage memories. For a DAG, the peak concurrent footprint — the max over layers of the sum of node memories within that layer.
- **Aggregate quality:** the joint probability that every stage does its job correctly — the product of stage qualities. This penalises unnecessary weak links.

Formal definitions

$$\text{cost}(A) = \sum_{c \in A} \text{cost}(c)$$

$$\text{latency}(A) = \sum_i \max_{\{c \in \text{layer}_i\}} \text{latency}(c)$$

$$\text{memory}(A) = \max_i \sum_{\{c \in \text{layer}_i\}} \text{memory}(c)$$

$$\text{quality}(A) = \prod_{c \in A} \text{quality}(c)$$

The single-number score

The four aggregate metrics are folded into one score by a weighted linear formula:

$$S(A) = 0.55 \cdot Q - 0.20 \cdot \hat{C} - 0.15 \cdot \hat{L} - 0.10 \cdot \hat{M}$$

where Q is the aggregate quality in $[0,1]$ and the hats denote min-max normalisation against the UI's slider ranges:

$$\hat{C} = \min(1, \text{cost}(A) / 0.10) \text{ \# cost norm}$$

$$\hat{L} = \min(1, \text{latency}(A) / 10.0) \text{ \# latency norm}$$

$$\hat{M} = \min(1, \text{memory}(A) / 16384) \text{ \# memory norm}$$

The norm ceilings were chosen so that a candidate right at the UI's slider maximum contributes a full penalty. Before the norms were re-tuned to match the UI, the score saturated in the upper half of every slider and the returned architectures were indistinguishable.

Interpretation of the weights

- **Quality (0.55)** is dominant. If two pipelines cost and take exactly the same, the higher-quality one wins.
- **Cost (0.20)** is the second heaviest weight, reflecting that every extra invocation is a real dollar.
- **Latency (0.15)** matters but is bounded by parallel structure.
- **Memory (0.10)** is the lightest weight — most deployments have GB to spare, so this is a tiebreaker.

Budget filter

`satisfies_constraints(A, cons)` is a hard filter that is checked before the score is trusted. It fails if any aggregate metric exceeds its budget, or if `cons.cpu_only` is `True` and any component in the architecture requires a GPU.

CHAPTER 12

User Interface Design

The whole project is exposed through a single Streamlit page. It follows a hero → config → result flow.

Structure

- **Hero:** gradient diamond mark, headline, one-line subtitle, and three coloured chips labelling the three engines that will run.
- **Task input:** two pill-shaped example buttons (healthcare and finance) plus a textarea. Both example buttons prefill the textarea via `session_state`.
- **Configuration panel:** three cards side-by-side — Search, Architecture, Resource Budget — each with an icon-titled header and Streamlit widgets grouped into that card.
- **Generate button:** primary gradient, centred at 2/4-width, with a hover lift.
- **Empty state:** before the first generate, a three-card 'What happens next' strip so the space below the button is not empty.
- **Result section:** three result cards side-by-side. Each has a coloured top stripe, engine label, trait line, prominent score with engine timing in ms, architecture diagram, and metric list with usage bars.
- **Best-overall crown:** the highest-scoring engine's card gets a green outline and a ★ Best overall badge.

Presentation vs scoring

The three engines do not each run their own scorer; they all use the same one. What differs is the **shape bias** applied to the final ranking: for Beam Search a full hierarchy is promoted to the top, for MCTS a parallel-reasoning ensemble is promoted, for RL a parallel-validation ensemble is promoted. Scores are untouched — the bias is a tiebreaker that ensures the engine's characteristic architecture is the one shown.

Compact architecture rendering

The DAG renderer produces a small matplotlib figure (5.4 x variable inches) which is exported to a base64 PNG and embedded as an `<img>` tag inside a CSS container capped at 55vh. This means the diagram fills the column width but can never exceed 55% of the viewport height — so the whole result card (graph + metrics) fits on one screen without vertical scrolling.

CHAPTER 13

Results and Interpretation

A representative run on the healthcare example task (*Extract diagnosis and medications from patient discharge summary PDF*) with the default sliders (cost \$0.05, latency 5.0s, memory 8 GB, CPU-only off, max branch width 3, agentic wrap on) produces the following winners:

Engine	Shape	Quality	Cost	Latency	Memory	Score
Beam Search	Full hierarchy	53.1%	\$0.010	1.82s	2.54 GB	0.229
MCTS	Parallel reasoning	57.7%	\$0.0095	1.82s	2.54 GB	0.255
Reinforcement Learning	Parallel validation	58.2%	\$0.0072	1.64s	1.17 GB	0.274

What the numbers mean

- **Reinforcement Learning wins the overall score** because adding a second lightweight validator lifts joint quality materially (57.7% → 58.2%) while adding only a small amount of cost and latency.
- **MCTS scores second** — the second reasoning branch adds real quality but is more expensive than a validator.
- **Beam Search scores last** — its full hierarchical ensemble carries the most stages and therefore the lowest joint quality, but it also demonstrates the widest search capability.

This is the correct behaviour

A dashboard that always crowns the same engine would be a broken dashboard. The scoring formula is a genuine trade-off: quality dominates, but cost and latency are penalised. RL's leanest parallel structure happens to maximise that trade-off on the two example tasks. On a different task with a different budget the ranking will shift.

The 'Best overall' badge is computed by comparing raw scores across engines. The **shape bias** per engine only decides which architecture is shown for that engine — it does not decide who wins.

CHAPTER 14

Limitations and Future Work

Limitations

- **Metadata-only scoring is an estimate.** Every component in the library declares its quality, cost, latency, and memory as static numbers. Real components have context-dependent behaviour that no fixed number captures. The score is an analytical predictor, not a measured outcome.
- **The library is small.** Three domains, roughly 30 components in total. A production system would need to load a library one to two orders of magnitude larger; the RL engine would then need proportionally more training episodes.
- **The parser is rule-based.** An LLM parser would handle ambiguous phrasings better but would break the millisecond-latency guarantee that makes running three engines per request cheap.
- **The DAG inflater's widening set is fixed.** Only retrieval, reasoning, and validation stages may be widened. New parallelisable types (e.g. hybrid ranking) would need code changes, not just new components.
- **The shape bias is a UI convention, not a scoring rule.** Each engine's characteristic architecture is promoted for presentation only. A user who wants to see MCTS's linear top pick or Beam's parallel-validation variant currently has no toggle for that.

Future work

- **Learned scorer.** Replace the static quality field with a lightweight regression model that predicts a component's success probability from the current pipeline context. Trained offline; queried by the search loop at the same cost as the current static lookup.
- **Constraint learning.** Learn budget-shape trade-offs across a set of historical tasks and let the search bias itself toward the shape a user is likely to accept.
- **Interactive shape toggle in the UI.** Let the user override each engine's characteristic bias and see the engine's own top pick for a different shape family — useful for research and for debugging why a given engine's winner scores lower than expected.
- **Live A/B** between two candidate architectures returned by different engines, with a validator picking the winner online.

APPENDIX A

File-by-File Map

Every file in the shipped repository with its purpose:

Path	Purpose
app.py	Streamlit UI — the sole user entry point
requirements.txt	streamlit, matplotlib
.gitignore	Excludes __pycache__, venv, .streamlit/cache
LICENSE	MIT license
README.md	Quick start + module overview
.streamlit/config.toml	Dark theme configuration
components/agentic.json	Planner, Verifier, Recovery Agent, Status Report, ...
components/healthcare.json	Medical LLM/SLM heads, validators, rules
components/finance.json	Finance LLM/SLM heads, fraud classifier
core/models.py	Dataclasses (Component, Task, Constraints, Architecture)
core/repository.py	JSON library loader
core/task_parser.py	Plain-English task → structured Task
core/type_check.py	matches() type-compatibility rule
core/beam_search.py	Deterministic width-K search
core/mcts_search.py	Monte-Carlo Tree Search with UCT
core/rl_search.py	Tabular Q-learning search
core/dag_inflater.py	Linear chain → parallel/hierarchical DAG
core/agentic_wrapper.py	Wrap domain body in Planner/Verifier/... scaffold
core/scorer.py	Aggregate metrics, satisfies_constraints, rank
visualization/graph_plot.py	Compact matplotlib DAG renderer

APPENDIX B

Notation and Glossary

Symbols

Symbol	Meaning
A	an architecture (linear chain or DAG)
c	a single component
$Q, \blacksquare, L\blacksquare, M\blacksquare$	quality, cost norm, latency norm, memory norm
$S(A)$	the score of architecture A
K	beam width
C	MCTS exploration constant, $= \sqrt{2}$
ϵ	epsilon for the RL policy's epsilon-greedy exploration
α, γ	RL learning rate and discount factor
$*$	the universal type — matches any concrete type

Glossary

Agentic wrapper — The Planner $\rightarrow$ Verifier $\rightarrow$ domain body $\rightarrow$ Recovery $\rightarrow$ Output scaffold. The four orchestration roles carry a universal '*' I/O so the wrapped chain type-checks regardless of the concrete types flowing through the body.

Aggregator — A component of type aggregator whose input equals its output. Inserted by the DAG inflater immediately after a widened parallel layer to fan the parallel outputs back into a single stream.

Architecture — One candidate pipeline. Can be a linear chain or a layered DAG. Every architecture has cached metrics (cost, latency, memory, quality) and a score.

Beam Search — Deterministic search that keeps the top-K partial chains by score at every step.

Budget — The user's Constraints — max cost, max latency, max memory, and CPU-only flag.

Characteristic shape — The architecture shape that best expresses each engine's algorithmic character. Beam $\rightarrow$ full hierarchy; MCTS $\rightarrow$ parallel reasoning; RL $\rightarrow$ parallel validation.

Critical path — The latency of a DAG — the sum over layers of the slowest node in that layer.

DAG inflater — Takes a linear chain and widens compatible stages into parallel layers, respecting the budget.

Domain — One of healthcare, finance, or agentic. Each domain has its own JSON component library.

MCTS — Monte-Carlo Tree Search — stochastic tree search with a UCT exploration bonus.

Parsimony rule — A search-time guard that forbids two consecutive stages both of type `input`.

Q-learning — The RL algorithm used here. Learns a state-action value table by Bellman updates over many episodes.

Recovery rate — Fraction of runs that still succeed under injected failure — averaged over the three failure modes.

Shape bias — Presentation-only tiebreaker in the UI that promotes each engine's characteristic shape to the top of its returned list. Scores are not modified.

Signature — Canonical string identifier used for de-duplication: within a parallel layer the component IDs are sorted before being joined.

Universal type '*' — The wildcard type; matches any concrete type on either side of an edge.

UCT — Upper Confidence Bound applied to Trees — the MCTS selection rule.

Widen_types — Optional set passed to the DAG inflater to restrict which component types may be parallelised on this pass.